

MACHINE LEARNING

Muhammad Atif Saeed (Lecturer DS & AI)

What is the KNN Classification Algorithm?

- KNN (K-Nearest Neighbors) is a simple, non-parametric method for classification.
- Given a set of labeled data points, the KNN classification algorithm finds the k data points in the training set that are closest to the point to be classified. Then, it assigns the label that is most common among those k data points.
- Here, we need to specify the number of nearest neighbors, k , which is a user-specified parameter. The basic idea behind the KNN algorithm is that similar data points will have similar labels.

K-Nearest Neighbor Classification Algorithm

- The inputs to the algorithm are:
 - The dataset with labeled data points
 - The number k i.e. the number of nearest neighbors that we use to find the class of any new instance of data.
 - The new data point.

K-Nearest Neighbor Classification Algorithm

- First, we choose the number k and a distance metric. You can take any distance metric such as Euclidean, Minkowski, or Manhattan distance for numerical attributes in the dataset. You can also specify your own distance metric if you have datasets having categorical or mixed attributes.
- For a new data point P , calculate its distance to all the existing data points.
- Select the k -nearest data points, where k is a user-specified parameter.
- Among the k -nearest neighbors, count the number of data points in each class. We do this to select the class label with a majority of data points in the k neighbors that we select.
- Assign the new data point to the class with the majority class label among the k -nearest neighbors.

Distance Functions

- There are various methods for calculating the distance between the new point and each training point
- The most commonly known methods are:
 - Euclidian (for continuous)
 - Manhattan (for continuous)
 - Hamming distance (for categorical)

Euclidean	$\sqrt{\sum_{i=1}^k (x_i - y_i)^2}$
Manhattan	$\sum_{i=1}^k x_i - y_i $
Minkowski	$\left(\sum_{i=1}^k (x_i - y_i)^q \right)^{1/q}$

KNN Example (Cont.)

Point	Coordinates	Class Label
A1	(2,10)	C2
A2	(2, 6)	C1
A3	(11,11)	C3
A4	(6, 9)	C2
A5	(6, 5)	C1
A6	(1, 2)	C1
A7	(5, 10)	C2
A8	(4, 9)	C2
A9	(10, 12)	C3
A10	(7, 5)	C1
A11	(9, 11)	C3
A12	(4, 6)	C1
A13	(3, 10)	C2
A15	(3, 8)	C2
A15	(6, 11)	C2

In the dataset, we have fifteen data points with three class labels. Now, suppose that we have to find the class label of the point $P = (5, 7)$.

For this, we will first specify the number of nearest neighbors i.e. k . Let us take k to be 3. Now, we will find the distance of P to each data point in the dataset.

For this KNN classification numerical example, we will use the euclidean distance metric. The following table shows the euclidean distance of P to each data point in the dataset.

KNN Example (Cont.)

Point	Coordinates	Distance from P (5, 7)
A1	(2, 10)	4.24
A2	(2, 6)	3.16
A3	(11, 11)	7.21
A4	(6, 9)	2.23
A5	(6, 5)	2.23
A6	(1, 2)	6.40
A7	(5, 10)	3.0
A8	(4, 9)	2.23
A9	(10, 12)	7.07
A10	(7, 5)	2.82
A11	(9, 11)	5.65
A12	(4, 6)	1.41
A13	(3, 10)	3.60
A15	(3, 8)	2.23
A15	(6, 11)	4.12

After finding the distance of each point in the dataset to P, we will sort the above points according to their distance from P (5, 7). After sorting, we get the following table.

K-Means Example (Cont.)

Point	Coordinates	Distance from P (5, 7)
A12	(4, 6)	1.41
A4	(6, 9)	2.23
A5	(6, 5)	2.23
A8	(4, 9)	2.23
A15	(3, 8)	2.23
A10	(7, 5)	2.82
A7	(5, 10)	3
A2	(2, 6)	3.16
A13	(3, 10)	3.6
A15	(6, 11)	4.12
A1	(2, 10)	4.24
A11	(9, 11)	5.65
A6	(1, 2)	6.4
A9	(10, 12)	7.07
A3	(11, 11)	7.21

As we have taken $k=3$, we will now consider the class labels of three points in the dataset nearest to point P to classify P. In the above table, A12, A4, and A5 are the closest 3 neighbors of point P. Hence, we will use the class labels of points A12, A4, and A5 to decide the class label for P.

Now, point A12, A4, and A5 have the class labels C1, C2, and C1 respectively. Among these points, the majority class label is C1. Therefore, we will specify the class label of point P = (5, 7) as C1. Hence, we have successfully used KNN classification to classify point P according to the given dataset.

Advantages of the KNN Classification

- **Simple to implement:** KNN is a simple and easy-to-implement classification algorithm that requires no training.
- **Versatility:** KNN can be used for both classification and regression problems. Whether you need to perform binary classification or multi-class classification, the K-nearest neighbor algorithm works well. By defining distance metrics for mixed and categorical data, you can also use KNN for the classification of categorical and mixed data types.
- **Non-parametric:** The KNN algorithm does not make any assumptions about the underlying data distribution, so it is well-suited for problems where the decision boundary is non-linear.
- **Adaptive:** KNN can adapt to changes in the training data, making it suitable for dynamic or evolving systems. It doesn't remember the pattern in the previous dataset. While classification, it calculates the instantly and then produces the results. Hence, for dynamic systems with changing datasets, KNN will work well.

Advantages of the KNN Classification

- **Multi-class:** You can use KNN classification to implement multi-class classification tasks. It works in a similar manner as binary classification and no extra calculations are required.
- **No Training:** We don't need to train the KNN classifier. It only stores the data points and uses them for classifying new instances of data.
- **Handling missing values:** KNN is less sensitive to missing values because the missing values can simply be ignored when calculating the distance.
- **Handling noisy data:** KNN is robust to noisy data and can handle irrelevant or redundant features since it focuses on the k closest neighbors. While calculating the class label for a data point, it used the majority of the class labels of k nearest neighbors. Hence, the noise in the data will become a minority class and won't affect the classification process.
- **Handling outliers:** KNN can be robust to outliers since the decision is based on the majority class among k -nearest neighbors.

Disadvantages of the KNN Classification

- **Computationally expensive:** KNN has a high computation cost during the prediction stage, especially when dealing with large datasets. The algorithm needs to calculate the distance between the new data point and all stored data points for each classification task. This can be slow and resource-intensive.
- **Memory-intensive:** KNN stores all the training instances. This can require a large amount of memory while dealing with large datasets.
- **Sensitive to irrelevant features:** KNN is sensitive to irrelevant or redundant features since it uses all the input features to calculate the distance between instances. Therefore, you first need to perform data preprocessing and keep only relevant features in the dataset.
- **Quality of distance function:** The effectiveness of the KNN algorithm greatly depends on the choice of the distance function. The algorithm may not work well for all types of data. For instance, we generally use dissimilarity measures for categorical data. In such a case, finding the majority class will become difficult as many of the data points will lie at the same distance from the new data point.

Disadvantages of the KNN Classification

- **High dimensionality:** KNN may not work well when the number of features is high. The curse of dimensionality can make it hard to find meaningful patterns in the data.
- **Need to determine the value of k:** The value of k is a user-specified parameter that needs to be determined through trial and error or using techniques such as cross-validation, which can be time-consuming. A small value of k could lead to overfitting as well as a big value of k can lead to underfitting.
- **Not good with categorical variable:** KNN is not good when the categorical variable is involved. It works well when the variable is numerical. However, you can define distance measures for categorical and mixed data types to perform KNN classification.
- **Slow prediction:** KNN is slow in prediction as it needs to calculate the distance of the new point from each stored point. This is a slow process and computationally expensive.

Applications of the KNN Classification

- **Image recognition:** We can use KNN classification to classify images based on their content, such as recognizing handwritten digits or identifying objects in an image.
- **Medical diagnosis:** We can use the KNN algorithm in medical diagnosis to classify patients based on their symptoms or medical history.
- **Recommender systems:** The recommender systems primarily use KNN classification to make recommendations based on the similarity between users or items.
- **Credit scoring:** Banking applications can use the KNN classification algorithm to classify loan applicants based on their credit history.
- **Speech recognition:** You can use the KNN algorithm to classify speech sounds and recognize spoken words.

Applications of the KNN Classification

- **Handwriting recognition:** KNN can be used to recognize handwriting and convert it into digital text.
- **Quality control:** You can use the KNN classification algorithm to classify items as defective or non-defective based on their features.
- **Gene expression analysis:** KNN can be used to classify genes based on their expression levels.
- **Bioinformatics:** KNN can be used to classify proteins based on their structural and functional characteristics.
- **Natural Language Processing (NLP):** You can use the KNN algorithm for text classification, such as sentiment analysis, spam detection, and topic classification.